

Motion Planning in Urban Environments via Self-Play Reinforcement Learning

Islem Kobbi¹, Tiago Rocha Gonçalves^{1,2} and Fawzi Nashashibi¹

Abstract—This work presents a self-play reinforcement learning framework for mid-to-mid motion planning in urban autonomous driving. Unlike mid-to-end approaches that output low-level control commands, the proposed planner maps structured observations to continuous trajectories parameterized by cubic Hermite splines, preserving the modular perception-planning-control pipeline standard in industry. Training is conducted entirely through self-play in PufferDrive, a GPU-accelerated multi-agent simulator in which all agents share and continuously update a single policy, generating progressively challenging interactions without manual scenario design. Four neural architectures are evaluated under identical training conditions: an MLP baseline, an MLP encoder with LSTM processing, a Transformer encoder with MLP processing, and a combined Transformer-LSTM model. On 150 unseen urban maps, the best policy achieves 92.2% task completion with a 6.9% collision rate. Across all architectures, recurrent memory and attention-based scene encoding both improve upon the MLP baseline. Recurrent memory provides the most favorable performance/efficiency trade-off, sustaining 340k environment steps per second while delivering the strongest completion and safety metrics, and is identified as the key architectural component for robust generalization under self-play dynamics.

Keywords: Self-Play Reinforcement Learning, Mid-to-Mid Motion Planning, Urban Autonomous Driving, Proximal Policy Optimization, Transformer, LSTM.

I. INTRODUCTION

Urban motion planning in dense city traffic requires reasoning about complex multi-agent interactions while meeting safety and comfort constraints. In such settings, rule-based systems and classical optimization pipelines often demand substantial engineering effort and still struggle to generalize across different types of road layouts and traffic behaviors [1], [2]. Reinforcement learning (RL) provides an alternative in which policies are learned from interaction, enabling adaptation to dynamics that are difficult to capture with explicitly programmed rules [3].

Despite this promise, RL is known to require a large number of samples; effective training commonly requires millions to billions of environment transitions. Simulator throughput therefore becomes a primary bottleneck. Physics-based simulators such as CARLA [4] and MetaDrive [5] provide high realism but are computationally expensive,

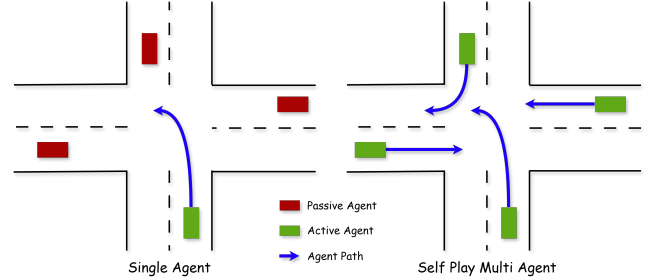


Fig. 1. Multi-agent self-play training vs. single-agent training.

limiting training speed. Data-driven GPU-accelerated simulators, including GPUdrive [6] and PufferDrive [7], mitigate this bottleneck by replaying logged real-world trajectories directly on a GPU, reaching 300k to 4 million steps per second while preserving realistic traffic distributions and maps. This level of throughput makes large-scale multi-agent RL training feasible.

Beyond simulation speed, the training outcome depends critically on the behavior of surrounding agents. Scripted background traffic provides limited interaction diversity and can yield brittle policies that overfit to predictable opponents [8]. As shown in Fig. 1, self-play addresses this limitation by having all agents execute a shared, continuously updated policy. As the policy improves, interactions become progressively more challenging, creating an automatic learning progression of multi-agent scenarios [9] without manual scenario design. GigaFlow [10] demonstrated this combination at scale, training urban driving policies purely through self-play with no expert demonstrations.

Existing self-play driving systems focus on mid-to-end policies that map structured observations directly to low-level control commands (steering and throttle) [10], [11]. While effective, this formulation merges planning and control into a single opaque policy, reducing modularity and complicating integration into industry-standard decision-making pipelines. Mid-to-mid approaches instead predict a continuous planned trajectory that is tracked by a separate high-level controller, preserving planning and control as distinct, transparent modules. This paradigm is better suited to industrial deployment, yet self-play RL has not been studied for mid-to-mid planners.

This paper addresses this gap by applying self-play RL to a mid-to-mid motion planner for urban multi-agent environments. At each decision step, the planner outputs a trajectory parameterized by Hermite splines [12]

¹I. Kobbi, T. Rocha Gonçalves and F. Nashashibi are with the ASTRA Team, National Institute for Research in Digital Science and Technology (INRIA), 75013 Paris, France. islem.kobbi@inria.fr, tiago.rocha@inria.fr, fawzi.nashashibi@inria.fr

²T. Rocha Gonçalves is also with Valeo Brain, 6 rue Daniel Costantini, 94000, Créteil, France. tiago.rocha@valeo.com

and is trained without expert demonstrations. Four neural architectures are evaluated under identical self-play conditions: an MLP baseline, an LSTM to measure the effect of recurrent memory, a Transformer encoder to measure the effect of attention-based scene encoding, and a combined Transformer-LSTM model. In addition, a comparison with behavior cloning quantifies the improvement provided by self-play RL over imitation learning alone.

The main contributions are

- A self-play RL framework for mid-to-mid motion planning in urban multi-agent environments, extending self-play beyond mid-to-end control.
- A revised reward format to stabilize training and ensure consistent performance across all planning objectives.
- A comparative evaluation of four neural architectures trained under identical self-play conditions, highlighting the role of recurrent memory and attention-based scene encoding on the model performance.
- A behavior-cloning ablation that quantifies the benefit of self-play RL relative to imitation learning alone.

II. RELATED WORK

A. RL-Based Motion Planning

Reinforcement learning has been explored for autonomous driving with different action abstractions. Mid-to-end methods map structured observations directly to low-level control commands (typically steering and acceleration), including recent large-scale self-play systems such as GigaFlow [10] and smaller-scale baselines such as CaRL [11]. While such methods are conceptually simple, they merge planning and control into a single policy, reducing modularity and interpretability.

Mid-to-mid methods instead predict a planned trajectory that is tracked by a separate low-level controller. Prior work has investigated trajectory-level planners with reinforcement learning in simpler settings such as highway driving [12]. However, the combination of self-play RL with mid-to-mid trajectory outputs in dense multi-agent urban environments remains relatively unexplored.

B. Self-Play in Reinforcement Learning

Self-play trains agents against their own evolving policies, creating an automatic learning progression as opponents improve alongside the learner. The approach has been highly effective in multi-agent domains, most notably in AlphaGo [9], where progressively stronger self-generated opponents enabled robust policy improvement without manual opponent design. Recent work has extended self-play to autonomous driving at scale: Robust Autonomy Emerges from Self-Play [10] demonstrates mid-to-end driving behavior learned purely from self-interaction in simulation without expert demonstrations. GPUdriVe [6] similarly targets large-scale multi-agent training with a data-driven simulator, although both lines of work primarily evaluate direct control action outputs.

C. Neural Architectures and Scene Encoding

Urban driving observations contain a variable number of surrounding agents and map elements, motivating permutation-invariant encoders. MLP-based encoders with pooling are computationally efficient but treat all entities uniformly. Recurrent architectures such as LSTMs provide temporal memory across decision steps, supporting more stable behavior. Transformer-based encoders use attention to dynamically weight entities and have shown strong performance in large-scale motion prediction benchmarks (e.g., Wayformer [13]). The present work compares MLP, LSTM, and Transformer-based encoders under the same self-play training setup to isolate architectural effects.

D. Simulation Environments

Simulators for autonomous driving RL broadly fall into physics-based and data-driven categories. Physics-based simulators such as CARLA [4] and MetaDrive [5] provide realistic vehicle dynamics with unlimited randomly generated synthetic maps but typically offer limited training throughput, which can be prohibitive for large-scale RL. Data-driven GPU-accelerated simulators such as GPUdriVe [6] improve throughput by replaying logged real-world scenarios in parallel on GPU and applying physics only to active agents, enabling large-scale multi-agent training. PufferDrive [7] builds on this direction and is designed for high-throughput self-play. In this work, PufferDrive is paired with a trajectory-tracking controller to support mid-to-mid (trajectory-level) action outputs.

III. MID-TO-MID MOTION PLANNER DESIGN

One goal of this work is to preserve the modularity of the standard autonomous-driving pipeline. As shown in Fig. 2, the motion planner takes as input a structured observation produced by the perception module stack together with a reference path from the route planner. It outputs a trajectory, i.e., a continuous path with an associated speed profile, which is then executed by the controller.

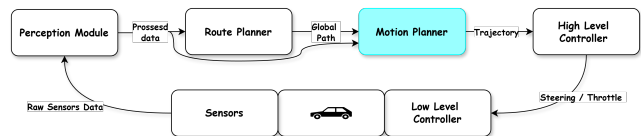


Fig. 2. System architecture overview.

A. Observation Space

The agent receives a fixed-size observation vector of 1138 values. The vector is divided into three groups: ego state, road users, and road elements, and all quantities are expressed in the ego-vehicle frame and normalized to $[-1, 1]$. **Ego State:** This group describes the ego vehicle and its short-horizon navigation context. It contains three components:

- *The ego state:* encodes the current speed, heading (represented by sine and cosine to avoid angle wrap-around),

the relative position of the current navigation goal, and the vehicle's width and length.

- *The reference path*: shown as green dots in Fig. 3, contains five equally spaced waypoints over the next 25 m of the route planner output in the ego frame. It depends only on the map (no interaction reasoning) and does not include a target speed profile.
- *The last action*: provides the previous action, giving the policy a minimal form of temporal context about its most recent intent.

Road Users: This group represents dynamic agents in the scene. It encodes up to 31 nearest road users (vehicles, pedestrians, cyclists, etc.) in the ego frame. Each road user is described by relative position, speed, heading (cos/sin), and bounding box dimensions.

Road Elements: This group captures the static layout of the road network. It encodes up to 128 road segments in the ego frame, each with position, heading (cos/sin), bounding box dimensions, and a semantic type (e.g., lane centerline, lane boundary, road edge, crosswalk). As shown in Fig. 3, rather than including every visible segment, segments along the reference path are selected to focus the representation on the region most relevant for near-term planning while keeping the observation size bounded.

For road users and road elements, entities are inserted based on their distance to the reference path until either the end of the observable area is reached or the observation vector is full, with any remaining slots filled with zeros.

TABLE I
OBSERVATION SPACE COMPONENTS AND DIMENSIONS.

Category	Component	Max	Dim	Total
Ego State	Ego State	1	7	7
	Ref Path	5	2	10
	Last Action	1	8	8
Road Users	Partners	31	7	217
Road Elements	Road	128	7	896
Total				1138

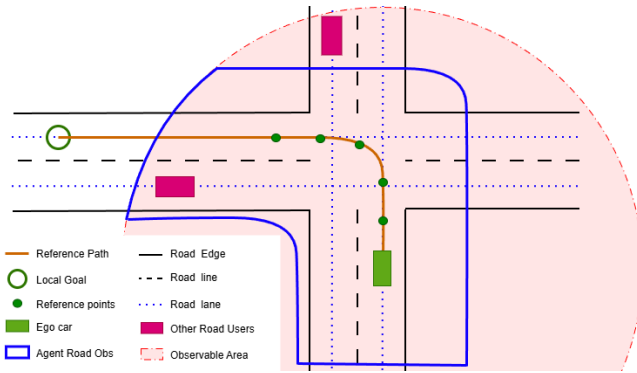


Fig. 3. Observation space structure and components.

B. Action Space

The action space is designed to produce continuous, jerk-free trajectories that respect vehicle dynamics while remaining interpretable; the output represents physically meaningful

control points rather than abstract function parameters. To achieve this, the agent outputs two control points, which are then interpolated using cubic Hermite splines, an approach previously validated for autonomous driving in [12] where it demonstrated strong effectiveness for smooth trajectory generation.

The agent outputs two control points P_1 and P_2 , with P_0 fixed at the ego vehicle's current state. Each control point is described by a position (x, y) expressed relative to the preceding point, a heading h , and a reference speed v , giving the following output vector:

$$\mathbf{a} = [x_1, y_1, h_1, v_1, x_2, y_2, h_2, v_2] \quad (1)$$

where the three control points are defined as

$$\begin{aligned} P_0 &= (0, 0, 0, v_0) && \text{(current ego state)} \\ P_1 &= (x_1, y_1, h_1, v_1) && \text{(relative to } P_0) \\ P_2 &= (x_2, y_2, h_2, v_2) && \text{(relative to } P_1) \end{aligned}$$

The agent outputs a normalized vector in $[-1, 1]$, which is then decoded into physical quantities as follows. The longitudinal position X is decoded as:

$$X = x \cdot (2 + v_{\text{current}}) \quad (2)$$

making the planning horizon adaptive to the vehicle's current speed; at low speeds the control points are placed closer, while at higher speeds the horizon extends further ahead. The lateral position y is denormalized to $[-10\text{m}, 10\text{m}]$, the heading h to $[-\pi/2, \pi/2]$, and the reference speed v to $[0 \text{ m/s}, 25 \text{ m/s}]$.

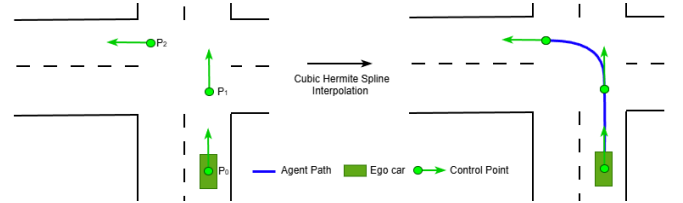


Fig. 4. Action space and trajectory generation.

C. Reward Function

The reward function is structured into two components: a *Path Reward* evaluated on the planned trajectory before execution and an *Execution Reward* accumulated during the trajectory tracking sub-steps. This separation allows the agent to be rewarded for the quality of its planning decisions independently of low-level controller tracking errors.

1) *Path Reward*: The path reward R_{path} is made up of four terms and is calculated only on the planned spline prior to execution.

Path Similarity: This leverages the availability of the human (logged) trajectory in a data-driven simulator. To encourage human-like behavior, a path similarity score is computed between the agent's planned path and the logged human path.

$$R_{\text{similarity}} = \frac{1}{12} \sum_{i=1}^{12} \begin{cases} 1 - \frac{\|P_{h,i} - P_{\text{exp},i}\|}{3} & \text{if } \|P_{h,i} - P_{\text{exp},i}\| < 3 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

where $P_{h,i}$ is one of the 12 equidistant points from the Hermite path and $P_{exp,i}$ is the closest point from the expert path.

Lane Alignment: This term encourages the agent to plan trajectories that are geometrically consistent with the road geometry, i.e., staying as close as possible to the center lane.

$$R_{lane} = \frac{1}{12} \sum_{i=1}^{12} \begin{cases} 1 - \frac{\|P_{h,i} - CL_i\|}{3} & \text{if } \|P_{h,i} - CL_i\| < 3 \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

where $P_{h,i}$ is one of the 12 equidistant points from the Hermite path and CL_i the closest point from the road centerline.

Efficiency: This term rewards the agent for maintaining a steady speed close to the reference speed provided by the route planner, without being overly aggressive. It is computed to peak at the reference speed and decreases if the agent is slower or faster than the reference speed.

$$R_{eff} = \begin{cases} 1 - \frac{V_{ref} - V}{V_{ref}}, & \text{if } V < V_{ref}, \\ 1 - \frac{V - V_{ref}}{V_{ref}}, & \text{if } V_{ref} \leq V \leq 2V_{ref}, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Where V is the mean speed of the two control points, and V_{ref} is the reference speed set by the route planner.

Smoothness: This term penalizes curved paths and encourages the agent to plan a smooth path by encouraging the lateral position and heading of the control points P_1 and P_2 to be as close as possible to 0.

$$R_{sm} = 1 - \frac{(|y_1| + |y_2|) + 4(|h_1| + |h_2|)}{10} \quad (6)$$

where y_1, y_2, h_1, h_2 are from the raw action of the agent.

The final path reward is the product of all terms. In preliminary experiments, this formulation encouraged the agent to improve all objectives simultaneously, whereas a more common weighted sum often led to over-optimizing some terms over the others depending on the chosen weights and early-stage exploration dynamics.

$$R_{path} = R_{similarity} \times R_{lane} \times R_{eff} \times R_{sm} \quad (7)$$

Note that all terms are by design normalized between 0 and 1.

2) *Execution Reward:* The execution reward R_{exe} is computed during the trajectory tracking phase, evaluated at each controller sub-step, and accumulated before the agent makes its next planning decision. It is composed of two terms.

Safety: Large negative penalties are triggered whenever the ego vehicle collides with another agent or leaves the drivable area during execution. Both events immediately terminate the episode:

$$R_{safety} = \begin{cases} -C_{col} & \text{if a collision occurs} \\ -C_{out} & \text{if the vehicle leaves the road} \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

where C_{col} and C_{out} are large constant penalty coefficients. These hard penalties ensure that safety constraints are never traded off against other reward components.

Success. A large positive bonus is awarded when the ego vehicle arrives within the goal radius, defined as the endpoint of the current route segment:

$$R_{success} = \begin{cases} +C_{goal} & \text{if } \|p_{ego} - p_{goal}\|_2 < r_{goal} \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

Where r_{goal} is the acceptance radius and C_{goal} a large positive constant. This term provides a clear and unambiguous signal for task completion.

The total step reward is:

$$R_{step} = C_{path} R_{path} + \sum_{i=1}^{N_{sub}} R_{exe}^i \quad (10)$$

where N_{sub} is the number of controller sub-steps taken while tracking the planned trajectory before the next agent decision.

IV. NEURAL NETWORK ARCHITECTURES

Four neural network architectures are evaluated in this work. As shown in Figure 5, all models follow the same three-stage pipeline:

- *Encoding stage:* Encodes the observation vector into a lower-size embedding.
- *Processing stage:* Processes the encoded embedding.
- *Decoding stage:* Converts the processed data into an action vector and value estimate.

The architectures differ only in the encoding and processing stages, enabling a controlled comparison.

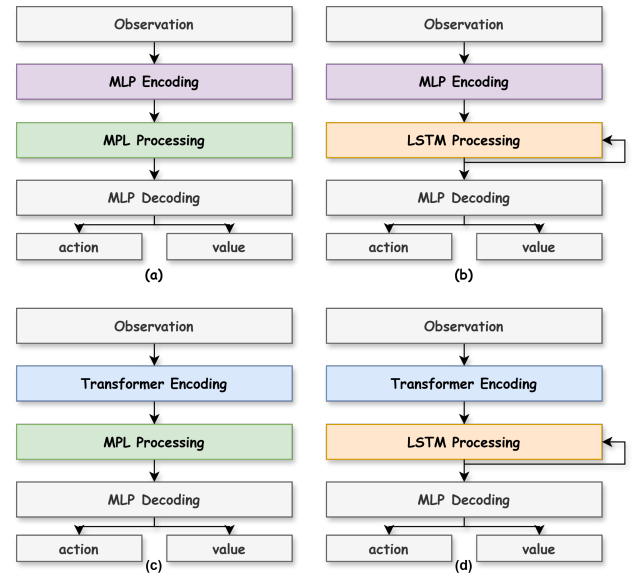


Fig. 5. Neural network architecture comparison: (a) MLP+MLP, (b) MLP+LSTM, (c) Trans+MLP, and (d) Trans+LSTM.

A. MLP+MLP: MLP Encoder with MLP Processing

The MLP baseline uses MLPs for both encoding and processing. Separate MLPs process the ego state, road users, and road elements, max-pooling to select a subset of each to produce fixed-size vectors that are concatenated and passed through a shared MLP to form the encoded representation. A lightweight MLP processing block then feeds the decoder. This model provides a simple, efficient reference for comparison.

B. MLP+LSTM: MLP Encoding with LSTM Processing

This variant keeps the MLP encoder but replaces the processing block with an LSTM. The encoded vector is input at each step, with the hidden state carried across timesteps to provide temporal memory. Comparing it to the MLP baseline isolates the effect of recurrence.

C. Trans+MLP: Transformer Encoding with MLP Processing

This model replaces the MLP encoder with a Transformer while keeping MLP processing. Entities (ego, road users, road elements) are embedded into a shared 256-D space with learnable type embeddings and passed through a Transformer encoder (8 heads, GELU, pre-norm). Mean pooling yields a fixed-size vector for the MLP processing and decoder. This isolates the contribution of attention-based encoding versus the MLP-based one.

D. Trans+LSTM: Transformer Encoding with LSTM Processing

This architecture combines Transformer encoding with LSTM processing. The Transformer produces a context-aware embedding at each step, and the LSTM maintains temporal memory over the episode. It is the most expressive configuration, capturing both spatial interactions and temporal dynamics.

V. TRAINING FRAMEWORK

A. Multi-Agent Environment Setup

Training is conducted in PufferDrive [7], a GPU-accelerated, data-driven simulator developed by Emergin Labs and designed for large-scale multi-agent self-play. Each of the 1,000 maps, derived from the Waymo Open Dataset, represents an urban road network populated with multiple agents. All agents are controlled by the same shared policy.

B. PPO Training Configuration

All four architectures are trained using identical PPO hyperparameters, ensuring that any performance differences are attributable solely to architectural choices. To ensure repeatability, each experiment is run three times with different random seeds. The hyperparameters and reward coefficients are summarized in Table II.

TABLE II
TRAINING AND ENVIRONMENT HYPERPARAMETERS.

	Parameter	Value
Train	Learning rate (α)	2×10^{-3}
	Discount factor (γ)	0.98
	GAE lambda (λ)	0.95
	Entropy coefficient	0.005
	Batch size	786,432
	Update epochs	1
Reward	Collision penalty	-10
	Off-road penalty	-10
	Goal reward	+10
	C_{path}	1

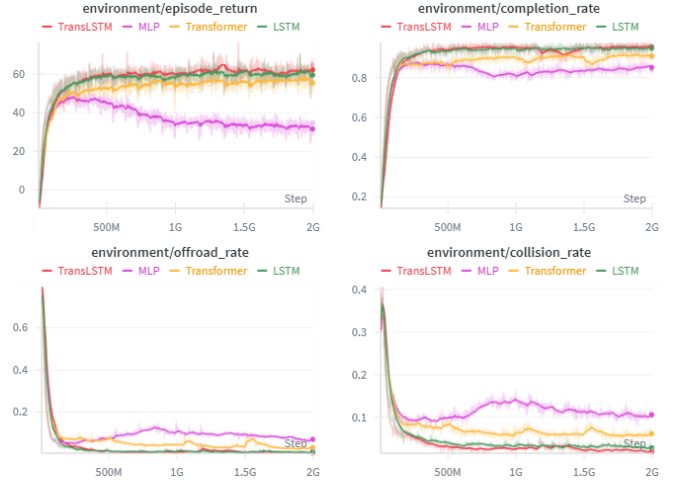


Fig. 6. Training curves over 2 billion steps.

VI. EXPERIMENTAL RESULTS

The four architectures are trained under identical self-play PPO training over 2 billion steps. Training curves are shown in Figure 6 and validation results on 150 unseen maps in Table III.

Trans+LSTM, MLP+LSTM, and Transformer all converge to returns of approximately 60 with completion rates above 0.90. The MLP improves initially but degrades beyond 500M steps, ending with the lowest return and highest collision and off-road rates.

On the unseen test maps, the MLP+LSTM achieves the best completion rate (**0.922**) and lowest collision rate (**0.069**), along with the highest path smoothness (**0.977**) and path reward (**0.659**). Trans+LSTM leads on episode return (**49.2**) and path alignment (**0.874**), while remaining highly competitive on path metrics. The Transformer shows strong overall balance but is outperformed by the LSTM on safety-critical metrics. The MLP continues to lag across all key metrics, confirming the advantage of recurrent and attention-based architectures under self-play dynamics.

A. Qualitative Results

Figure 7 illustrates the trajectories planned by the MLP+LSTM model across various urban scenarios.

Consistent with the quantitative results, where the differences between variables were negligible, the quality of the planned paths did not show significant variation across

TABLE III

TEST RESULTS ON UNSEEN MAPS (MEAN $\pm$ STD OF THE TOP-3 CHECKPOINTS PER ARCHITECTURE, RANKED BY LOWEST COLLISION RATE AND HIGHEST COMPLETION RATE).

Architecture	Return $\uparrow$	Completion $\uparrow$	Collision $\downarrow$	Offroad $\downarrow$	Alignment $\uparrow$	Smoothness $\uparrow$	SPS $\uparrow$	Params
MLP+MLP	39.5 $\pm$ 9.3	0.900 $\pm$ 0.003	0.084 $\pm$ 0.014	0.032 $\pm$ 0.010	0.868 $\pm$ 0.006	0.915 $\pm$ 0.017	380k	70k
MLP+LSTM	46.9 $\pm$ 1.6	0.922 $\pm$ 0.006	0.069 $\pm$ 0.005	0.027 $\pm$ 0.001	0.863 $\pm$ 0.005	0.977 $\pm$ 0.018	340k	596k
Trans+MLP	47.5 $\pm$ 3.0	0.907 $\pm$ 0.007	0.071 $\pm$ 0.001	0.033 $\pm$ 0.009	0.866 $\pm$ 0.008	0.972 $\pm$ 0.013	58k	177k
Trans+LSTM	49.2 $\pm$ 0.9	0.889 $\pm$ 0.017	0.081 $\pm$ 0.008	0.048 $\pm$ 0.013	0.874 $\pm$ 0.006	0.976 $\pm$ 0.003	53k	703k

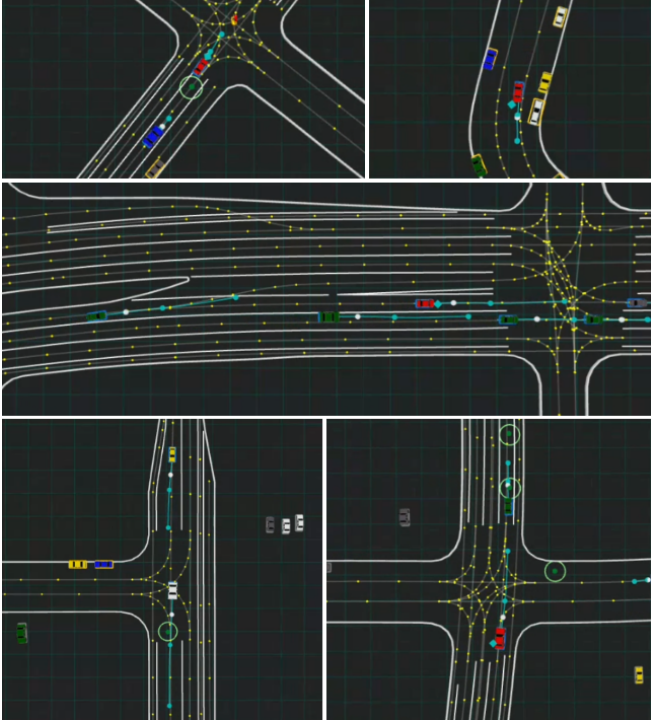


Fig. 7. Qualitative trajectories planned by the MLP+LSTM model across various urban scenarios. The model successfully navigates intersections and maintains flow while reacting to road geometry.

the different architectures. Qualitative behavior remained consistent across all models; each demonstrated the ability to react dynamically to road elements, effectively modulating speed and planning horizons based on traffic density and intersection complexity. Notably, a common behavior observed across all agents was a tendency to cut corners during turns to optimize path efficiency within the learned cost map.

VII. ABLATION STUDIES

This section compares three training variants using the MLP+LSTM architecture: self-play with the proposed product reward (SP product), self-play with a weighted-sum reward (SP sum), and imitation learning via behavior cloning (IL BC). Results are detailed in Table IV.

The importance of the multiplicative reward structure is demonstrated by replacing Eq. 7 with a weighted sum:

$$R_{\text{path}}^{\text{sum}} = 0.2R_{\text{similarity}} + 0.3R_{\text{lane}} + 0.3R_{\text{eff}} + 0.2R_{\text{sm}}. \quad (11)$$

While this formulation yields slight improvements in geometric alignment (0.896) and similarity (0.806), it leads to

a sharp decline in functional performance. Completion rates drop to 0.657, while the Did Not Finish (DNF) rate climbs to 0.245. This suggests a reward misalignment where the agent prioritizes geometric metrics over actual goal completion. In contrast, the product formulation ensures joint optimization, where a failure in any single aspect significantly penalizes the total reward.

The IL BC model achieves the highest path similarity (0.871), accurately mimicking expert trajectories. However, it fails to generalize during deployment, resulting in a low completion rate (0.238) and high collision (0.267) and off-road (0.506) rates. These results underscore the limitations of imitation learning due to distributional shifts in complex urban environments.

Ultimately, self-play with the product reward provides the most robust performance, achieving the highest completion rate (0.922), zero DNFs, and the best safety profile.

TABLE IV

ABLATION STUDY OF THE MLP+LSTM ARCHITECTURE COMPARING SELF-PLAY WITH PRODUCT REWARD (SP PRODUCT), SELF-PLAY WITH WEIGHTED-SUM REWARD (SP SUM), AND BEHAVIOR CLONING (IL BC).

Metric	SP (product)	SP (sum)	IL BC
Completion $\uparrow$	0.922	0.657	0.238
Collision $\downarrow$	0.069	0.077	0.267
Offroad $\downarrow$	0.027	0.033	0.506
DNF $\downarrow$	0.000	0.245	0.197
Path Alignment $\uparrow$	0.863	0.896	0.894
Path Similarity $\uparrow$	0.775	0.806	0.871
Path Efficiency $\uparrow$	1.000	0.307	0.434
Path Smoothness $\uparrow$	0.977	0.971	0.820

VIII. DISCUSSION

The experimental results validate the proposed self-play RL framework for mid-to-mid trajectory planning. The best-performing model achieves over 92% task completion on unseen maps while maintaining high path quality, confirming that self-play RL is a robust alternative to both classical planning pipelines and imitation-based methods. Crucially, the self-play curriculum—where all agents simultaneously execute and improve a shared policy—allows the planner to encounter progressively challenging multi-agent interactions without manual scenario engineering. This automatic difficulty scaling is a primary driver of the observed generalization.

A central finding is the importance of temporal context. Both LSTM and Transformer-based models consistently outperform the MLP baseline, indicating that recurrent memory

and attention mechanisms help the agent balance competing reward objectives—such as reaching the goal while maintaining smooth, consistent trajectories—rather than collapsing onto a single dominant term. While the Trans+LSTM configuration achieves the highest training return, its slightly lower test completion rate compared to MLP+LSTM suggests that its increased model capacity may require further regularization or greater data diversity to fully close the generalization gap.

The multiplicative reward structure proved essential to this balance. By taking the product of all path reward terms, the formulation penalizes the neglect of any single objective, enforcing joint satisfaction across all criteria. Ablation studies using a weighted-sum alternative showed that agents over-optimized geometric metrics at the expense of task completion, leading to a sharp rise in DNF episodes. This behavior persisted across various weight combinations, which typically resulted in either convergence to a sub-optimal solution or unstable training. However, the relative weighting of individual terms remains an open design choice; current equal weighting can encourage smoothing behaviors, such as corner-cutting, which trades strict lane adherence for improved smoothness scores. Future work should explore non-linear weightings, such as *log* or *exp* functions, to better balance reward terms.

From a computational standpoint, the LSTM block adds only modest overhead compared to the MLP (340k vs. 380k steps per second). In contrast, Transformer-based architectures run at approximately one-seventh of the MLP’s throughput, calling into question their value relative to their high computational cost. Given that MLP+LSTM achieves the best completion and safety metrics, it represents the most practical architecture for large-scale self-play training, combining strong generalization with efficient simulation.

IX. CONCLUSIONS

This paper introduces a self-play reinforcement learning framework for mid-to-mid urban motion planning, in which a shared policy is trained across all agents simultaneously in a GPU-accelerated multi-agent simulator. Self-play provides an automatic curriculum of progressively challenging interactions, removing the need for manual scenario design or expert demonstrations. The resulting planner achieves over 92% task completion on unseen maps with strong path quality, substantially outperforming behavior cloning, which reaches only 23.8% completion in the same closed-loop evaluation. This large gap demonstrates that self-play RL is the decisive ingredient for robust generalization under distribution shift in complex urban environments.

Among the four architectures evaluated, recurrent memory is the most impactful component. The MLP+LSTM architecture achieves the best completion and collision rates while remaining computationally efficient at 340k steps per second, making it the recommended configuration for deployment in a modular planning pipeline. Attention-based encoding provides complementary benefits and can be considered when additional model capacity is available, noting that it

currently operates at one-seventh the throughput of the MLP. The ablation study further confirms that the multiplicative reward formulation is essential: it enforces joint optimization across all planning objectives and eliminates the reward misalignment observed with weighted-sum alternatives.

Future work will focus on three directions: (i) calibrating the relative weights of path reward components to further reduce emergent shortcut behaviors, (ii) extending the simulator to support traffic lights, road signs, and pedestrian crossings for more complete urban coverage, and (iii) benchmarking alternative RL algorithms beyond PPO against standardized driving evaluation protocols.

REFERENCES

- [1] B. Paden, M. Cáp, S. Z. Yong, D. S. Yershov, and E. Frazzoli, “A survey of motion planning and control techniques for self-driving urban vehicles,” *CoRR*, vol. abs/1604.07446, 2016. [Online]. Available: <http://arxiv.org/abs/1604.07446>
- [2] E. Schmerling, K. Leung, W. Vollprecht, and M. Pavone, “Multimodal probabilistic model-based planning for human-robot interaction,” *CoRR*, vol. abs/1710.09483, 2017. [Online]. Available: <http://arxiv.org/abs/1710.09483>
- [3] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. A. Sallab, S. K. Yogamani, and P. Pérez, “Deep reinforcement learning for autonomous driving: A survey,” *CoRR*, vol. abs/2002.00444, 2020. [Online]. Available: <https://arxiv.org/abs/2002.00444>
- [4] A. Dosovitskiy, G. Ros, F. Codevilla, A. M. López, and V. Koltun, “CARLA: an open urban driving simulator,” vol. abs/1711.03938, 2017. [Online]. Available: <http://arxiv.org/abs/1711.03938>
- [5] Q. Li, Z. Peng, Z. Xue, Q. Zhang, and B. Zhou, “Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning,” vol. abs/2109.12674, 2021. [Online]. Available: <https://arxiv.org/abs/2109.12674>
- [6] S. Kazemkhani, A. Pandya, D. Cornelisse, B. Shacklett, and E. Vinit-sky, “GPUDrive: Data-driven, multi-agent driving simulation at 1 million FPS,” Aug. 2024, p. arXiv:2408.01584.
- [7] D. Cornelisse*, S. Cheng*, P. Mandavilli, J. Hunt, K. Joseph, W. Doulazmi, V. Charraut, A. Gupta, J. Suarez, and E. Vinit-sky, “PufferDrive: A fast and friendly driving simulator for training and evaluating RL agents,” 2025. [Online]. Available: <https://github.com/Emerge-Lab/PufferDrive>
- [8] P. Karkus, M. Igl, Y. Chen, K. Chitta, J. A. Packer, B. Douillard, R. Tian, A. Naumann, G. Garcia-Cobo, S. Tan, A. Degirmenci, A. Popov, N. Smolyanskiy, U. Muller, B. Ivanovic, and M. Pavone, “Beyond behavior cloning in autonomous driving: a survey of closed-loop training techniques,” [Online]. Available: <https://api.semanticscholar.org/CorpusID:283728300>
- [9] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, “Mastering the game of go without human knowledge,” *Nature*, vol. 550, no. 7676, pp. 354–359, 2017. [Online]. Available: <https://doi.org/10.1038/nature24270>
- [10] M. Cusumano-Towner, D. Hafner, A. Hertzberg, B. Huval, A. Petrenko, E. Vinit-sky, E. Wijmans, T. Killian, S. Bowers, O. Sener, P. Krähenbühl, and V. Koltun, “Robust Autonomy Emerges from Self-Play,” *arXiv e-prints*, p. arXiv:2502.03349, Feb. 2025.
- [11] B. Jaeger, D. Dauner, J. Beßwenger, S. Gerstenecker, K. Chitta, and A. Geiger, “CaRL: Learning Scalable Planning Policies with Simple Rewards,” Apr. 2025, p. arXiv:2504.17838.
- [12] I. Kobbi, H. Atoui, and F. Nashashibi, “Reinforcement Learning for Mid-to-Mid Motion Planning in Autonomous Driving,” in *ITSC 2025 - The IEEE International Conference on Intelligent Transportation Systems*, Gold Coast, Australia, Nov. 2025. [Online]. Available: <https://hal.science/hal-05294346>
- [13] N. Nayakanti, R. Al-Rfou, A. Zhou, K. Goel, K. S. Refaat, and B. Sapp, “Wayformer: Motion forecasting via simple and efficient attention networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 2980–2987.